



VIT[®]
BHOPAL

VIT BHOPAL UNIVERSITY
CSA2001 – Fundamental in AI and ML

PROJECT REPORT

Real Time Autonomous Navigation System

Smart navigation in changing environments using adaptive pathfinding algorithms.

<i>Name</i>	Siddharth Sharma
<i>Registration No.</i>	25BCE10977
<i>Branch / Program</i>	B.Tech - CSE CORE
<i>Year</i>	First Year (2025–26)
<i>Course Code</i>	CSA2001 — Fundamental in AI and ML
<i>Date of Submission</i>	March 31, 2026

1. Project Overview

This project presents a **Real-Time Autonomous Navigation System** that simulates a delivery agent navigating a grid-based environment. The system implements and benchmarks three classical search algorithms — Breadth-First Search (BFS), Uniform Cost Search (UCS), and A* — while supporting both static and moving obstacles. A dynamic replanning loop allows the agent to adapt its route on the fly whenever its planned path becomes blocked.

The codebase is structured across three Python modules, with a Pygame-based interactive visualizer providing real-time feedback on agent movement, path planning, and cost accumulation.

- Grid environment with customisable size and obstacle density
- Static obstacles (#) and dynamic obstacles (D) that move each time step
- Three pathfinding algorithms compared across four map configurations
- Real-time Pygame visualizer with cost/time HUD

2. Problem Statement & Objectives

Traditional static pathfinding fails in environments where obstacles can appear or move unpredictably — a common scenario in real-world delivery and autonomous vehicle routing. The core challenge is maintaining both optimality and responsiveness as the world changes.

Objectives:

- Design a grid environment that supports both static and dynamic (moving) obstacles.
- Implement BFS, UCS, and A* with a Manhattan-distance heuristic.
- Enable the agent to detect path blockages and replan in real time.
- Benchmark algorithms on execution time, path cost, and nodes expanded across map sizes.
- Provide an interactive Pygame visualizer for intuitive inspection of navigation behaviour.

3. System Architecture

The system is divided into three modules, each with a single clear responsibility:

File	Role
agent_pathfinding_env.py	Core engine: Grid, Agent, BFS, UCS, A*, SimulationController, map generator
main.py	Experiment runner: map configs, algorithm benchmarking, dynamic simulation, Pygame launch
pygame_ui.py	Real-time visualizer: renders grid, path, agent, cost/time overlay via Pygame
README.md	Installation guide, feature list, project structure overview

File	Role
statement.md	Problem statement, objectives, approach, challenges & learning outcomes

The layered design ensures that algorithm logic (`agent_pathfinding_env.py`) is fully independent of presentation (`pygame_ui.py`), making it easy to swap the visualizer or add new algorithms without touching core search code.

4. Environment Modeling

4.1 Grid & Cells

The environment is a 2-D grid of **GridCell** objects. Each cell stores its (x, y) coordinates, terrain type, movement cost, and two boolean flags: `is_obstacle` and `is_dynamic_obstacle`. Terrain types and their default costs are:

Symbol	Meaning	Cost
.	Open cell	1
#	Static obstacle	inf
S	Start position	1
G	Goal position	1
D	Dynamic obstacle	1
1/2/3	Weighted terrain	1 / 2 / 3

4.2 Dynamic Obstacles

Dynamic obstacles are represented by **DynamicObstacle** objects. Each obstacle carries an (x, y) position and a `movement_pattern` — a list of (dx, dy) deltas that it cycles through each simulation step. The grid's `update_dynamic_obstacles_positions()` method clears the old cell flag, advances the obstacle, and marks the new cell, keeping the grid state consistent for downstream pathfinding.

4.3 Map Generation

`generate_map_string()` procedurally creates maps of arbitrary width/height with configurable obstacle density. Start and goal positions are either specified explicitly or drawn randomly from unoccupied cells. Dynamic obstacle positions can also be injected. The function returns a plain-text map string that `Grid.from_map_string()` parses into a live Grid object.

5. Pathfinding Algorithms

All three algorithms share a common interface: they accept a **Grid**, start position, goal position (and optionally a heuristic), and return a tuple of (path, cost, nodes_expanded). Path reconstruction walks parent pointers from the goal node back to the start.

Algorithm	Strategy	Complete?	Optimal?	Time Complexity	Best Use
BFS	Breadth-first expansion	Yes	Yes*	$O(b^d)$	Unweighted, shallow goals
UCS	Cheapest-first expansion	Yes	Yes	$O(b^{(1+C^*/e)})$	Weighted grids, cost-sensitive
A*	$f(n)=g(n)+h(n)$ heuristic	Yes	Yes*	$O(b^d)$	Large maps, fast optimal path

5.1 Breadth-First Search (BFS)

Uses a **deque** as a FIFO queue. Expands nodes layer by layer, guaranteeing the minimum number of steps to the goal on unweighted grids. A visited dictionary tracks seen positions to prevent re-expansion. BFS does not account for terrain costs, so it may not return the cheapest path on weighted maps.

5.2 Uniform Cost Search (UCS)

Uses a **min-heap** (heapq) ordered by cumulative cost $g(n)$. Always expands the cheapest frontier node, guaranteeing optimal cost on weighted graphs. UCS is equivalent to Dijkstra's algorithm and naturally handles variable terrain costs.

5.3 A* Search

Extends UCS with an admissible heuristic $h(n)$. The priority $f(n) = g(n) + h(n)$ directs the search toward the goal, dramatically reducing nodes expanded on large maps. The Manhattan distance heuristic — $|x - x_{\text{goal}}| + |y - y_{\text{goal}}|$ — is admissible for 4-directional grid movement (no diagonals), ensuring optimality is preserved.

6. Dynamic Replanning

The **SimulationController** class orchestrates the agent–environment interaction loop:

- **Step 1 – Move obstacles:** `grid.update_dynamic_obstacles_positions()` advances every dynamic obstacle by one step in its movement pattern.
- **Step 2 – Collision check:** The controller inspects the next cell on the agent's planned path. If it is now blocked, a replan flag is set.
- **Step 3 – Replan (if needed):** The chosen search algorithm is invoked from the agent's current position. A fresh path replaces the stale one.
- **Step 4 – Move agent:** The agent advances one cell along the (potentially new) path, accumulating movement cost and incrementing its time counter.
- **Step 5 – Goal check:** If the agent reaches `goal_pos` the simulation terminates successfully.

Replanning is triggered proactively — the next step is inspected before the agent moves, not after a collision. This means the agent never enters a blocked cell.

7. Experiment Design & Metrics

The `conduct_full_experiment()` function runs every algorithm on every map configuration and collects structured results into a Pandas DataFrame. Seaborn bar charts (log-scale) then compare execution time and nodes expanded across conditions.

Map Size	Obstacles	Algorithm	Metric Tracked
5×5	~10%	BFS / UCS / A*	Time (ms), path length, nodes expanded
15×15	~20%	BFS / UCS / A*	Time (ms), path length, nodes expanded
25×25	~25%	BFS / UCS / A*	Time (ms), path length, nodes expanded
3×3 Dynamic	1 moving	A* (replanning)	Steps to goal, replan triggers, total cost

Key metrics recorded per run: **execution_time_ms** (wall-clock, via `time.perf_counter`), **path_cost** (sum of terrain costs along the path), **path_length** (number of steps), and **nodes_expanded** (proxy for search effort). Results are exported as a Markdown table via `results_df.to_markdown()`.

Expected Findings

- A* expands the fewest nodes on medium and large maps due to its heuristic guidance.
- BFS and A* return equal-length paths on unweighted cells; UCS matches when all costs are uniform.
- Execution time scales roughly as $O(W \times H)$ for BFS/UCS and sub-linearly for A* on open maps.
- Nodes expanded grows significantly with obstacle density as detours force deeper searches.

8. Pygame Visualization

The **GridVisualizer** class in `pygame_ui.py` renders the simulation at 30 px per cell with a 500 ms inter-step delay for legibility. Each entity is colour-coded:

Colour	Entity
Black	Static obstacle (#)
Green	Start cell (S)
Red	Goal cell (G)
Purple	Dynamic obstacle (D)
Blue	Agent current position
Yellow	Planned path cells
White	Open traversable cell

A HUD in the top-left corner displays cumulative cost and time step, updating every frame. The simulation loop is governed by `SimulationController.run_step()`, so the visualizer adds zero algorithm logic — it is purely a rendering layer.

9. Key Design Decisions

Modular separation: Core logic, experiment runner, and visualizer are in separate files. This allows unit-testing algorithms without a display and swapping the UI without touching search code.

Common algorithm interface: All three search functions return (path, cost, nodes_expanded), making it trivial to add new algorithms by dropping in a function with the same signature.

Proactive replanning: Checking the next step before moving (rather than after a collision) prevents the agent from ever occupying a blocked cell, which simplifies state management.

Map strings as interchange format: Using ASCII map strings decouples map design from the Grid class. Any text editor can create a map; `Grid.from_map_string()` handles parsing.

Admissible heuristic only: Manhattan distance is admissible for 4-directional movement, preserving A* optimality. Using an inadmissible heuristic would speed up search but sacrifice guaranteed shortest paths.

10. Challenges & Solutions

Dynamic state consistency: When an obstacle moves, both the old and new cells must be updated atomically. Solved by clearing `is_dynamic_obstacle` on the source cell before calling `obstacle.move()`, then setting it on the destination.

Replanning correctness: After obstacles move, the replanned path must start from the agent's current position (not the original start). The `SimulationController` always passes `agent.position` as the search start.

Pygame-simulation synchronisation: The visualizer must render the updated grid after obstacles move but before the agent moves. Achieved by calling `draw_grid()` and `draw_path()` each frame before `run_step()`.

Stuck-agent detection: If no path exists after replanning, `run_step()` returns `False` and the simulation halts cleanly rather than looping indefinitely.

11. Learning Outcomes

- Deep practical understanding of BFS, UCS, and A* — their data structures, correctness proofs, and performance trade-offs.
- Experience designing a modular simulation loop that separates sensing, planning, and acting.
- Hands-on use of Python's `heapq` and `collections.deque` for efficient frontier management.
- Understanding of heuristic admissibility and its role in guaranteeing A* optimality.
- Proficiency with Pandas and Seaborn for structured experiment logging and visualisation.
- Practical Pygame experience: event loops, frame timing, colour-coded rendering.

12. Future Improvements

D* Lite / LPA*: Incremental replanning algorithms that only recompute the parts of the path affected by changes, reducing replanning cost on large maps.

Multi-agent support: Extend the grid and simulation controller to handle multiple agents with collision avoidance (e.g., CBS — Conflict-Based Search).

AI-driven obstacle movement: Replace fixed movement patterns with goal-directed or learned behaviours so obstacles react to the agent.

Web dashboard: Replace Pygame with a browser-based dashboard (e.g., Dash or Streamlit) for easier sharing and remote monitoring.

Diagonal movement: Support 8-directional movement with a Chebyshev or Euclidean heuristic for more realistic navigation.

Weighted terrain experiments: Introduce terrain types 2 and 3 into benchmark maps to study how UCS and A* exploit cost information while BFS ignores it.

13. Conclusion

This project successfully implements and benchmarks three classical search algorithms in a dynamic grid environment. The proactive replanning loop, built around a clean `SimulationController` abstraction, demonstrates how autonomous agents can maintain goal-directed behaviour even as the environment changes beneath them.

A* with Manhattan distance consistently provides the best balance of speed and optimality on medium and large maps. BFS remains valuable for unweighted problems where simplicity is preferred. UCS is the correct choice whenever terrain costs vary and no reliable heuristic is available.

The Pygame visualizer transforms an abstract algorithm comparison into an intuitive, observable simulation — a pattern directly applicable to real-world robotics and delivery routing systems. The modular codebase provides a solid foundation for the future improvements outlined in Section 12.

Technologies used: Python 3 · NumPy · Pandas · Matplotlib · Seaborn · Pygame

Submitted as part of: Bring Your Own Project (BYOP) Assignment